

```

1 if (bakery has cannoli)
2     buy dessert; // bakery has cannoli
3 else if (bakery has apple turnovers && bakery has coffee) // no cannoli
4     buy dessert; // bakery has apple turnovers AND coffee

```

This pseudocode seems to work, but we must check for logical errors.

Using the numbered lines of pseudocode, let's trace the possibilities using a **trace table**.

has cannoli: line 1, condition is true -> line 2, buy dessert

no cannoli, no turnovers, no coffee: line 1, false -> line 3, false -> no dessert

no cannoli, yes turnovers, no coffee: line 1, false -> line 3, false -> no dessert

no cannoli, no turnovers, yes coffee: line 1, false -> line 3, false -> no dessert

no cannoli, yes turnovers, yes coffee: line 1, false -> line 3, true -> line 4, buy dessert

Moral of the story: This bakery had better get itself together.

There is a lot of pseudocode here! Controlling the flow of a program can be difficult and confusing, which is why it is a popular topic on the AP Exam. But it is also important because most programs, like most things we do in life, rely on conditions and react accordingly.

If we look at the bakery example one more time, the line

```
    buy dessert;
```

occurs twice. Good programming style attempts to repeat the same lines of code as little as possible, if ever. Therefore, we can rearrange the boolean operators in the following way, creating the same result:

```
if (bakery has cannoli OR (bakery has apple turnovers AND bakery has coffee))
```

```
    buy dessert;
```

That is a hefty boolean condition; however, it (1) eliminates the repetition of code and (2) provides a more “elegant” programming solution. “Elegant” is a relative term, of course, but the AP Exam often uses this subjective term to write code and to confuse you. Either way, you should be familiar with both ways.



Got a Question?

For answers to test-prep questions for all your tests and additional test-taking tips, subscribe to our YouTube channel at [youtube.com/ThePrincetonReview](https://www.youtube.com/ThePrincetonReview)



The AP Exam free-response questions do not require you to write code with “elegance”; in the free-response questions they will accept any code solution as long as it fulfills the specifications of the question.

To make things more complicated (or more life-like), consider a further idea. What if we want to execute *several* commands when a condition is true (or false) instead of just one? For example, using the bakery case, let’s say that buying cannoli is so exciting that we must devour it right away. In other words, if the conditions are met for the bakery having cannoli, we want to buy it AND eat it. The pseudocode would look something like:

```

1 if (bakery has cannoli)
2 {
3     buy dessert; // bakery has cannoli
4     eat dessert;
5 }
6 else if (bakery has apple turnovers && bakery has coffee) // no cannoli
7     buy dessert; // bakery has apple turnovers AND coffee
  
```

Use blocking to execute more than one statement based on a condition.

The { and } symbols in lines 2 and 5 indicate **blocking**, a technique used in flow control statements that allows the programmer to execute a series of commands (instead of just one) when a given condition is satisfied.

Here is a summary for evaluating boolean expressions:

- An && (and) expression is true if BOTH A and B are true.
- An || (or) expression is true if EITHER A or B is true, or if they are both true.
- The ! (not) operator simply reverses the truth value of the variable.

The truth value of an expression is often abbreviated to a single letter. Thus, A may represent an expression such as $x \geq 0$. The expression will either be true or false depending on the value of x, but for simplicity’s sake we just refer to the whole expression as A.

Consider the following three expressions:

To evaluate $A \ \&\& \ B$

First evaluate A. If A is false then stop: the whole expression is false. Since false $\&\&$ anything is false, there is no need to continue after the first false has been evaluated. This idea is called **short-circuit evaluation**. This idea is used (and tested) frequently. However, if A is true, then you must evaluate B to determine the truth value of the whole expression.

To evaluate $A \ || \ B$

First evaluate A. If A is true then stop: the whole expression is true. Since true $||$ anything is true, there is no need to continue after the first true has been evaluated. This is short-circuit evaluation again. However, if A is false, then you must evaluate B to determine the truth value of the whole expression.

To evaluate $!A$

First evaluate A. If A is true, the whole expression is false. If A is false, the whole expression is true.

1



Mark for Review

Consider the following code.

```
int x = 0;
if (x == 0)
    System.out.print("1");
else
    System.out.print("2");
    System.out.print("3");
```

Which of the following best describes the result of executing the code segment?

- (A) Since the value of x is 0, the first print statement will be performed, producing 1 as the output.
- (B) Since the value of x is 0, the first print statement will be performed, producing 13 as the output.
- (C) Since the value of x is 0, the first print statement will be performed, producing 123 as the output.
- (D) $==$ is not the correct boolean operator, so a syntax error will be produced by the compiler prior to execution.

Here's How to Crack It

Since `x` is assigned to 0, the condition of the `if` statement will be true, so the first print statement will be performed, and the output will be 1. The `else` statement will then be skipped, so 2 will not be outputted, eliminating (C). The trick here, however, is that the third print statement is NOT part of the `else` statement since it is not blocked with `{}`, even though it is (incorrectly) indented. This will output 3, eliminating (A). Furthermore, `==` is a valid boolean operator so (D) is incorrect (and a compiler will never produce a logical error). The correct answer is (B).

2**Mark for Review**

Consider the following boolean expression.

$$(X \ \&\& \ !Y) \ || \ (!X \ \&\& \ Y)$$

Which of the following condition(s) will always cause the expression to evaluate to true?

- ☐ (A) `X` is true
- ☐ (B) `Y` is false
- ☐ (C) `X` and `Y` have the same truth values
- ☐ (D) `X` and `Y` have opposite truth values

Here's How to Crack It

The `||` indicates that either $(X \ \&\& \ !Y)$ is true or $(!X \ \&\& \ Y)$ is true. Each of these expressions uses a combination of `X` and `Y`, thus ruling out (A) and (B). In the first expression, `X` is true while `Y` is false. In the second expression, the reverse is true: `Y` is true and `X` is false. This eliminates (C) because the truth values of `X` and `Y` cannot match. Since `X` and `Y` have opposite truth values, (D) is the correct answer.

To sum up, let's create a truth table of various combinations of boolean conditions, simply called `A` and `B`, and the truth possibilities based on those combinations. Since there are two variables, there are four possible combinations of their truth values: they are both true, only one of them is true (both ways), and neither is true. These possibilities are shown in the first two columns below. In subsequent columns, the truth values are shown for the boolean statements shown in the first row.

A	B	A && B	A B	!A	!B	!(A && B)	!A !B
T	T	T	T	F	F	F	F
T	F	F	T	F	T	T	T
F	T	F	T	T	F	T	T
F	F	F	F	T	T	T	T

Note that the truth values of the last two expressions are identical; these results are an illustration of De Morgan's Law, which might be thought of as the "distributive property" of boolean expressions. Applying a `!` (not) operator to an expression reverses the truth value of each variable and changes an `&&` to an `||`, or vice versa. If this law is applied to `!(A && B)`, the result is `!A || !B`, as shown in the last column.

The intricacies of `if` statements that we just explored will apply to all flow control statements, so understand them here before you move on to the next section.

Augustus De Morgan was a British mathematician and logician in the 19th century. De Morgan's Law can be used to simplify complicated boolean expressions. The basic concept is to know that an expression can be rewritten to simplify and logically hold the same value. For example, logically speaking:

`!true = false`

`!false = true`

So the two forms of the law are stated as follows:

1. `!(a && b)` is equivalent to `!a || !b`
2. `!(a || b)` is equivalent to `!a && !b`

In other words, when you apply a negation to an expression inside parentheses, negate each expression and swap the operator.

Applying De Morgan's Law:

Example 1

`!(x >= 0 && y < 0)`

First, handle the negation and swap `&&` to `||`

`!(x >= 0) || !(y < 0)`

Now simplify each expression.

- If x is NOT ≥ 0 then it must be < 0
- If y is NOT < 0 then it must be ≥ 0

$$x < 0 \parallel y \geq 0$$

Example 2

$$\neg(x == 0 \ \&\& \ \neg(y == 0))$$

First, handle the negation and swap $\&\&$ to $\parallel$

$$\neg(x == 0) \parallel \neg\neg(y == 0)$$

Now simplify each expression.

- If x is NOT $== 0$ then it must be $!= 0$
- Two negations eliminate each other

$$x != 0 \parallel y == 0$$

3 Mark for Review

Consider the following code segment.

```
boolean a = true, b = false;
if (/* missing code */)
    System.out.print("Nice job.");
else
    System.out.print("Nicer job.");
```

Which of the following could be used to replace `/* missing code */` so that the output of this block of code is "Nicer job."?

- I. `a && !b`
- II. `!a || b`
- III. `!a && b`

(A) I only

(B) I and II only

(C) I and III only

(D) II and III only

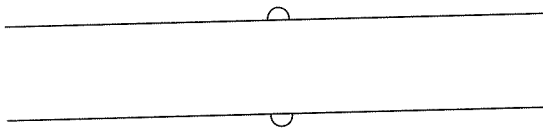
Here's How to Crack It

Ahh, the dreaded “Roman numeral” problems that the College Board loves to torture us with. If “Nicer job.” has to be displayed, then the condition must evaluate to false.

Create a truth table.

a	b	a && !b	!a b	!a && b
true	false	true && !false true && true true	!true false false false false	!true && false false && false false

Only the last two expressions result in a false outcome; thus (D) is the correct answer.



4



Mark for Review

Suppose p and q are declared as boolean variables and have been initialized to unknown truth values.

What does the following boolean expression evaluate to?

$(!p \ \&\& \ !q) \ || \ !(p \ || \ q)$

- (A) The expression always evaluates to true.
- (B) The expression always evaluates to false.
- (C) The expression evaluates to true whenever p is false.
- (D) The expression evaluates to false whenever p and q have opposite truth values.

Roman numeral problems are only as annoying as their hardest option; look at the options and do the easiest one first, which is often not I. Then use Process of Elimination.

Here's How to Crack It

Using De Morgan's Law, we can see that the truth value of the expression on the right, $!(p \parallel q)$, simplifies to the expression on the left, $!p \ \&\& \ !q$. Since trial and error is a useful strategy, set up a truth table to evaluate the potential answers.

P	q	$!p \ \&\& \ !q$	Eliminate (A) and (B) because sometimes the expression evaluated true and sometimes false. Eliminate (C): in the second row, p was false but the expression did not evaluate to true. Choice (D) is correct. Checking the first two rows, p and q had opposite truth values and both rows evaluated to false.
T	F	$!T \ \&\& \ !F$ $F \ \&\& \ T$ F	
F	T	$!F \ \&\& \ !T$ $T \ \&\& \ F$ F	
T	T	$!T \ \&\& \ !T$ $F \ \&\& \ F$ F	
F	F	$!F \ \&\& \ !F$ $T \ \&\& \ T$ T	

CHAPTER 5 KEY TERMS

if

conditional statement

flow control

condition

boolean operator ($==$)

truth value

compound condition

 $\&\&$ (logical *and*) $\parallel$ (logical *or*) $!$ (logical *not*) $==$ (is equal to) $!=$ (is not equal to)

trace table

blocking

short-circuit evaluation

CHAPTER 5 REVIEW DRILL

Answers to the review questions can be found in Chapter 12.

1 Mark for Review

Consider the following code segment:

```
for (int i = 200; i > 0; i /= 3)
{
    if (i % 2 == 0)
        System.out.print(i + " ");
}
```

What is the output as a result of executing the code segment?

(A) 200 66 22 7 2

(B) 66 22 7 2

(C) 200 66 22 2

(D) 7

2 Mark for Review

Suppose `p` and `q` are declared as boolean variables and have been initialized to unknown truth values.

What does the following boolean expression evaluate to?

```
(p && !q) || (p && q)
```

(A) The expression always evaluates to the same value as `p`.

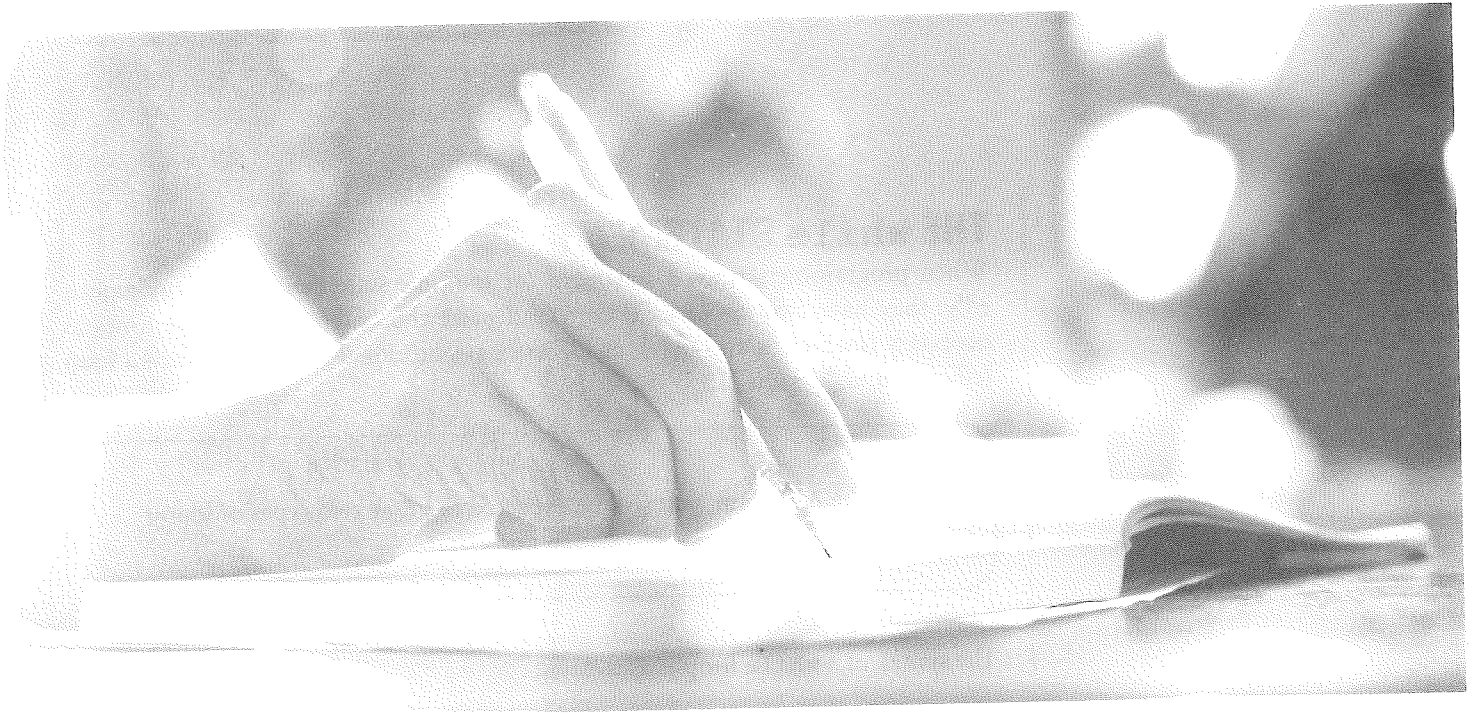
(B) The expression always evaluates to the same value as `q`.

(C) The expression always evaluates to `false`.

(D) The expression evaluates to `true` only if `p` and `q` have opposite truth values.

Chapter 5 Summary

- Values can be compared to form boolean statements using `==` (equals), `!=` (not equal), `<` (less than), `<=` (less than or equal to), `>` (greater than), and `>=` (greater than or equal to).
- Boolean statements can be combined using `&&` (and), `||` (or), and `!` (not).
- `If` statements can be used to allow a command or series of commands to be executed once only if a certain boolean statement is true. `Else` statements can be used to execute a different command if that condition is not met.



Chapter 6

Iteration

THE **while** STATEMENT

When programmers want a statement (or multiple statements) to occur repeatedly, they have two options: (1) copy and paste the statement as many times as necessary (inefficient and ugly), or (2) write a conditional statement called a **loop** (efficient and “elegant”).

On the AP Exam, you will be expected to know two types of loops:
`while` and `for`.

The syntax of a **while** loop is as follows:

```
while (condition) statement;
```

Note its similarity to the `if` statement. The `while` statement is a loop that does exactly what its name implies: the loop cycles again and again, *while* the condition is true. Consider a paint-soaked paintbrush that you just used to paint your bedroom. Hopefully, you will not store the paintbrush away until it is no longer dirty. A `while` loop that could represent this situation in pseudocode would look like:

```
1 while (paintbrush is dirty)
2     do not put away paintbrush;
3 put away paintbrush;
```

Note that if the condition is true (paintbrush is dirty), line 2 will continue to execute, over and over again, until the condition is false (paintbrush is not dirty). Once the condition is false, line 2 is skipped and line 3 is finally executed.

Unfortunately, this loop is fundamentally flawed. Although its logic *seems* to be correct, it is not. Within the statement(s) of a loop, the variable(s) in the condition must be modified so that there is a chance that the truth value of the condition will be changed. How will the paintbrush become less dirty—is this a self-cleaning paintbrush? Probably not. Therefore, the paintbrush will actually remain dirty, and Java will keep executing line 2, forever. When this happens, the programmer has written an **infinite loop**, which is a logical error and is therefore undesirable. Ever have your computer randomly “freeze” in the middle of your work, leaving it unresponsive and causing you to lose that perfectly sculpted digital picture that you just spent an hour perfecting? That is an infinite loop and definitely not desirable.

On the AP Exam, you must trace code in multiple-choice questions and detect infinite loops. In free-response questions, you must write code that is free of infinite loops.

While loops, like `if` statements, are therefore dependent on their conditions. If a condition is more intricate, as it usually is (Do we put the paintbrush away if it is not dirty, but it is wet? Not dirty but it is still not *clean*?), we can use the techniques outlined in the previous section. Boolean operators are mandatory and compound conditions and/or blocking are appropriate when necessary. A more realistic pseudocode representation of our paintbrush example could be

```

1  while (paintbrush is dirty && paintbrush is wet) // ! means NOT wet
2  {
3      clean paintbrush; // is it COMPLETELY clean?
4      dry paintbrush; // is it COMPLETELY dry?
5      do not put paintbrush away;
6  }
7  put paintbrush away;
```

In this example, the paintbrush will continue to be cleaned and dried (and not put away) until it is either NOT dirty or NOT wet, or both, at which point the condition will be false and the paintbrush will be put away. Remember that an `&&` condition is true only if both conditions are true; therefore, if the paintbrush is not dirty but is still wet (or vice versa), it will be put away. Is this the desired result? Who cares?! Reading code is not about logical understanding of a situation; rather, it is about understanding the code and its result.

It is also worth noting here that an `&&` statement may be **short-circuited**. Since both boolean statements on either side of the `&&` must be true, a false result from the first statement will automatically render the condition false. As a result, Java will completely skip the second condition, bypassing the rest of the condition.

1 Mark for Review

Consider the following code segment.

```
int val1 = 2, val2 = 22, val3 = 78;
while (val2 % val1 == 0 || val2 % 3 == 0)
{
    val3++;
    val2--;
}
```

What will val3 contain after the code segment is executed?

(A) 78

(B) 79

(C) 80

(D) None of the above

When tracing loops,
make a table of values for
each variable to organize
your work.

Here's How to Crack It

Tracing the loop is the best way to handle this type of problem. Remember that % returns the remainder of the division between the two numbers. In order for the condition to be true, there first must be no remainder when dividing val2 by val1. Since $22/2 = 11$, there is no remainder, and the condition will be true (an "or" statement will be true if its first condition is true), and the loop statements will execute, incrementing val3 to 79 and decrementing val2 to 21. Since there are no visible statements that would decrement val3 below 79, we can eliminate (A). Execution then returns to the condition; this time, we have $21/2$, which does yield a remainder, so we check the rest of the division, and $21 \% 3$ does not yield a remainder, so the condition is true overall and the loop statements execute again. We now have val3 = 80 and val2 = 20, eliminating (B). Again we try to evaluate the condition, which will be true since $20/2$ has no remainder, increasing val3 to 81 and decreasing val2 to 19. We can now eliminate (C). By Process of Elimination, (D) is the answer.

THE for STATEMENT

A `for` statement is another type of loop. **For loops** and **while loops** are equally effective in controlling the flow of a program when a statement (or set of statements) is to be executed repeatedly. The syntax is quite different, however:

```
for (initializer; condition; incremter) statement;
```

Since the `for` loop requires more components, it can potentially avoid an infinite loop situation, although there is no guarantee. Let's try a pseudocode example. Consider a younger sibling who is learning to count to 22. The child will start with 1 and continue counting until reaching 22, and then stop. The counting process repeats until the condition is met. Pseudocode may look like:

```
for (start with 1; not yet reached 22; go to the next number)
```

```
count the number;
```

The execution of a `for` loop is more difficult to understand. Here are the steps:

1. The initializer will occur first, beginning the process.
2. The condition will be checked to make sure it is true.
3. The statement(s) will execute.
4. The incremter will occur, changing the number.
5. See step (1) and keep repeating (1) to (4), until (2) is false.
6. The loop ends.

A code example of this situation, outputting the current number to the screen, could be:

```
for (int num = 1; num <= 22; num++)
```

```
System.out.println(num);
```

This code will produce each counting number and output it to the screen, each on a separate line. Once again, remember that boolean operators are mandatory and compound conditions and/or blocking are appropriate when necessary.

Take Note: `for` loops and `while` loops are interchangeable, although one method may look more elegant than the other.



Go Online!

For answers to test-prep questions for all your tests and additional test-taking tips, subscribe to our YouTube channel at [youtube.com/ThePrincetonReview](https://www.youtube.com/ThePrincetonReview)



For and while statements are loops; if statements are not.

On the AP Exam, multiple-choice questions may show you several loop structures in both formats and ask you questions that compare the results, so you should practice working between them. On a free-response question that asks you to write a loop, you will be equally correct whether you decide to write a for loop or a while loop.

The following two code examples involving loops have identical results:

```
int num = 1;
while (num <= 3)
{
    System.out.println(num);
    num++;
}
System.out.println("Done");
```

```
for (int num = 1; num <= 3; num++)
    System.out.println(num);
System.out.println("Done");
```

Each set of code is equally effective and equally respected in both the programming world and on the AP Exam. But multiple-choice questions can use either format, or both, so you should familiarize yourself with both.

While loops perform the blocked statement(s) once and then evaluate the condition to determine whether the loop continues; for loops evaluate the condition first and then perform the blocked statement(s) if the condition is initially true.

2

Mark for Review

What will be the output when the following code is evaluated?

```
for (int k = 0; k < 3; k++)  
{  
    for (int j = 1; j < 4; j++)  
    {  
        System.out.print(j + " ");  
    }  
    System.out.println();  
}
```

(A) 1 2 3 4
1 2 3 4
1 2 3 4

(B) 0 1 2
0 1 2
0 1 2
0 1 2

(C) 1 2 3
1 2 3
1 2 3

(D) 1 2 3
1 2 3
1 2 3
1 2 3

Here's How to Crack It

One glance at the condition in the outer `for` loop reveals that it will iterate a total of 3 times ($k = 0, 1$, and 2). The same will occur for the inner loop ($j = 1, 2$, and 3). Answers that involve loops of 4 can be eliminated, ruling out every choice except (C). There is no need to continue. However, to see why (C) works, work through the two loops. The outer loop sets $k = 0$, then $j = 1$. It prints 1 and a space and then increments j to 2. It prints 2 and a space and then increments j to 3. It prints 3 and a space and then increments j to 4. Since 4 is not less than 4, it terminates the inner loop. `System.out.println()` prints a line break. Then k is incremented to 1. Since k does not affect the output or the value of j , the loop again prints 1 2 3 before printing a line break. The value of k increments to 2, printing a third line of 1 2 3. Finally, k is incremented to 3, which terminates the outer loop, leaving (C) as the result of the print. The answer is (C).

3 Mark for Review

Consider the following code fragments. Assume `someNum` has been correctly defined and initialized as a positive integer.

```
I. int i;
   for (i = 0; i < someNum; i++)
   {
       someNum--;
   }

II. int i;
   for (i = 1; i < someNum - 1; i++)
   {
       someNum -= 1;
   }

III. int i = 0;
   while (i < someNum)
   {
       i++;
       someNum--;
   }
```

All the following statements are true about these code fragments EXCEPT

- (A) The for loops in I and II do not have the same number of iterations.
- (B) The for loop in I and the while loop in III have the same number of iterations.
- (C) The value of `i` after the execution of II and III is the same.
- (D) I, II, and III produce identical results in `someNum`.

Here's How to Crack It

For an EXCEPT question, cross out the original question and rewrite it without the negative so that it's easier to keep your task in mind. In this case, rewrite the question to say, "Which is false?"

The loops in I and III accomplish the same task. In each loop, `i` is initialized to 0, executes while `i < someNum`, then increments `i` by 1 and decrements `someNum` by 1 within the loop. Thus, any choice indicating the iterations or results of loops I and III are the same should be eliminated, because these conditions are true. Eliminate (B).

The loop in II will have fewer iterations than the loops in I and III (initializing `i` to 1 instead of 0, ending the loop when `i < someNum - 1` instead of `i < someNum`). As a result, the loop in II will be executed fewer times. Choice (A), indicating that I and II do not have the same number of iterations, is true, so eliminate it.

For the remaining choices, let `someNum` equal a positive (nonzero) value (not too big)...let's try 5.

I

i	SomeNum
0	5
1	4
2	3
3	2

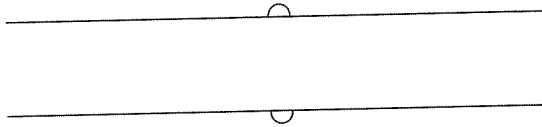
II

i	SomeNum	SomeNum - 1
1	5	4
2	4	3
3	3	2

III

i	SomeNum
0	5
1	4
2	3
3	2

From these trace tables of values, you can see that the final value of `i` after the execution of II and III is 3, demonstrating that (C) is true. Eliminate (C). The value of `someNum` is only the same for I and III, so (D) is false, and is the correct choice.



4



Mark for Review

Consider the following code segment:

```
for (int i = 1; i < 100; i = i * 2)
{
    if (i / 50 == 0)
        System.out.print(i + " ");
}
```

What is printed as a result of executing the code segment?

(A) 1 2 4 8 16 32 64

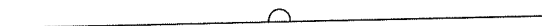
(B) 1 2 4 8 16 32

(C) 2 4 8 16 32 64

(D) 2 4 8 16 32

Here's How to Crack It

The initial condition for execution to enter the loop is `i = 1`. Since `1/50 = 0` in integer division, 1 will be displayed, so eliminate (C) and (D). The difference between (A) and (B) is the 64 at the end, so let's skip the output in between and focus on the 64. If `i = 64`, the `if` statement will be false, and thus 64 will not be printed. Therefore, the answer is (B).



**Don't Forget This!**

Head over to your free online student tools to download a printable version of this chart and other important pages from within this book.

Looking for More Help with Your APs?

We now offer specialized AP tutoring and course packages that guarantee a 4 or 5 on the AP. To see which courses are offered and available, visit princetonreview.com/college/ap-test-prep



Synopsis

Here's a quick synopsis of which concepts from the chapters so far you can expect to find on the exam, and which you won't.

	Concepts Covered on the AP Computer Science A Exam	Concepts Not Covered on the AP Computer Science A Exam
Primitives	• int • double • boolean	• short • long • byte • char • float
Increment/Decrement Operators	• x++ • x--	• ++x • --x
Logical Operators	• == • != • < • <= • > • >= • && • • !	• & • • ^ • << • >> • >>>
Conditional Statements	• if/else • for • while	• do/while • switch • plain and labeled break • continue
Miscellaneous		• ?: (ternary operator) • User input • JavaDoc comments

CHAPTER 6 KEY TERMS

loop
while loop
infinite loop
short-circuited
for loop

CHAPTER 6 REVIEW DRILL

Answers to the review questions can be found in Chapter 12.

1 Mark for Review

Consider the following output:

```
0 1
0 2 4
0 3 6 9
0 4 8 12 16
```

Which of the following code segments will produce this output?

(A)

```
for (int x = 1; x < 5; x++)
{
    for (int z = 0; z <= x; z++)
    {
        System.out.print(x * z + " ");
    }
    System.out.println(" ");
}
```

(B)

```
for (int x = 1; x <= 5; x++)
{
    for (int z = 0; z < x; z++)
    {
        System.out.print(x * z + " ");
    }
    System.out.println(" ");
}
```

(C)

```
for (int x = 1; x < 5; x++)
{
    for (int z = 0; z <= 4; z++)
    {
        System.out.print(x * z + " ");
    }
    System.out.println(" ");
}
```

(D)

```
for (int x = 1; x <= 5; x++)
{
    for (int z = 0; z <= x; z++)
    {
        System.out.print(x * z + " ");
    }
    System.out.println(" ");
}
```

2 Mark for Review

The speed limit of a stretch of highway is 55 miles per hour (mph). The highway patrol issues speeding tickets to anyone caught going faster than 55 miles per hour. The fine for speeding is based on the following scale:

Speed	Fine
greater than 55 mph but less than 65 mph	\$100
greater than or equal to 65 mph but less than 75 mph	\$150
greater than or equal to 75 mph	\$300

If the value of the `int` variable `speed` is the speed of a driver who was pulled over for going faster than 55 mph, which of the following code segments will assign the correct value to the `int` variable `fine`?

- I.

```
if (speed >= 75)
    fine = 300;
if (speed >= 65 && speed < 75)
    fine = 150;
if (speed > 55 && speed < 65)
    fine = 100;
```
- II.

```
if (speed >= 75)
    fine = 300;
if (65 <= speed < 75)
    fine = 150;
if (55 < speed < 65)
    fine = 100;
```
- III.

```
if (speed >= 75)
    fine = 300;
if (speed >= 65)
    fine = 150;
if (speed > 55)
    fine = 100;
```

☐ (A) I only

☐ (B) III only

☐ (C) I and II

☐ (D) I and III

3 Mark for Review

Consider the following code segment:

```
int x = 10;
int y = 3;
boolean b = true;
for (int i = 0; i < 15; i += 5)
{
    x = x + y;
    b = (x % y == 2);
    if (!b)
    {
        y++;
        i += 5;
    }
}
```

What is the value of `x` after the code segment executes?

(A) 10

(B) 15

(C) 17

(D) 22

4 Mark for Review

In the following statement, `a` and `b` are boolean variables:

```
boolean c = (a && b) || !(a || b);
```

Under what conditions will the value of `c` be `true`?

(A) Only when the value of `a` is different than the value of `b`

(B) Only when the value of `a` is the same as the value of `b`

(C) Only when `a` and `b` are both `true`

(D) The value of `c` will be `true` for all values of `a` and `b`.

5 Mark for Review

Consider the following code segment:

```
while ((x > y) || y >= z)
{
    //code not shown
}
```

In the code segment above, *x*, *y*, and *z* are variables of type `int`. Which of the following must be true after the code segment has executed?

(A) `x > y || y >= z`

(B) `x <= y || y > z`

(C) `x < y && y <= z`

(D) `x <= y && y < z`

6 Mark for Review

Consider the following code segment:

```
int a = 0;
for (int i = 0; i < 10; i++)
{
    for (int k = 0; k <= 5; k++)
    {
        for (int z = 1; z <= 16; z = z * 2)
        {
            a++;
        }
    }
}
```

What is the value of *a* after the code segment executes?

(A) 31

(B) 180

(C) 200

(D) 300

7 Mark for Review

Consider the following code segment:

```
int x = 10;  
int y = x / 3;  
int z = x % 2;  
x++;  
System.out.println(x)
```

What is printed as a result of executing the code segment above?

(A) 2

(B) 10

(C) 11

(D) 15

8 Mark for Review

Consider the following code segment:

```
int a = 10;  
double b = 3.7;  
int c = 4;  
int x = (int) (a + b);  
double y = (double) a / c;  
double z = (double) (a / c);  
double w = x + y + z;  
System.out.println(w);
```

What is printed as a result of evaluating the code above?

(A) 10

(B) 15.5

(C) 17

(D) 17.5

9

 Mark for Review

Consider the following code segments:

```
I.  int x = 10;
    int y = 20;
    int z = 0;
    if (x < y && 10 < y/z)
    {
        System.out.println("Homer");
    }
    else
    {
        System.out.println("Bart");
    }
```

```
II. int x = 10;
    int y = 20;
    int z = 0;
    if (x > y && 10 < y/z)
        System.out.println("Homer");
    else
        System.out.println("Bart");
```

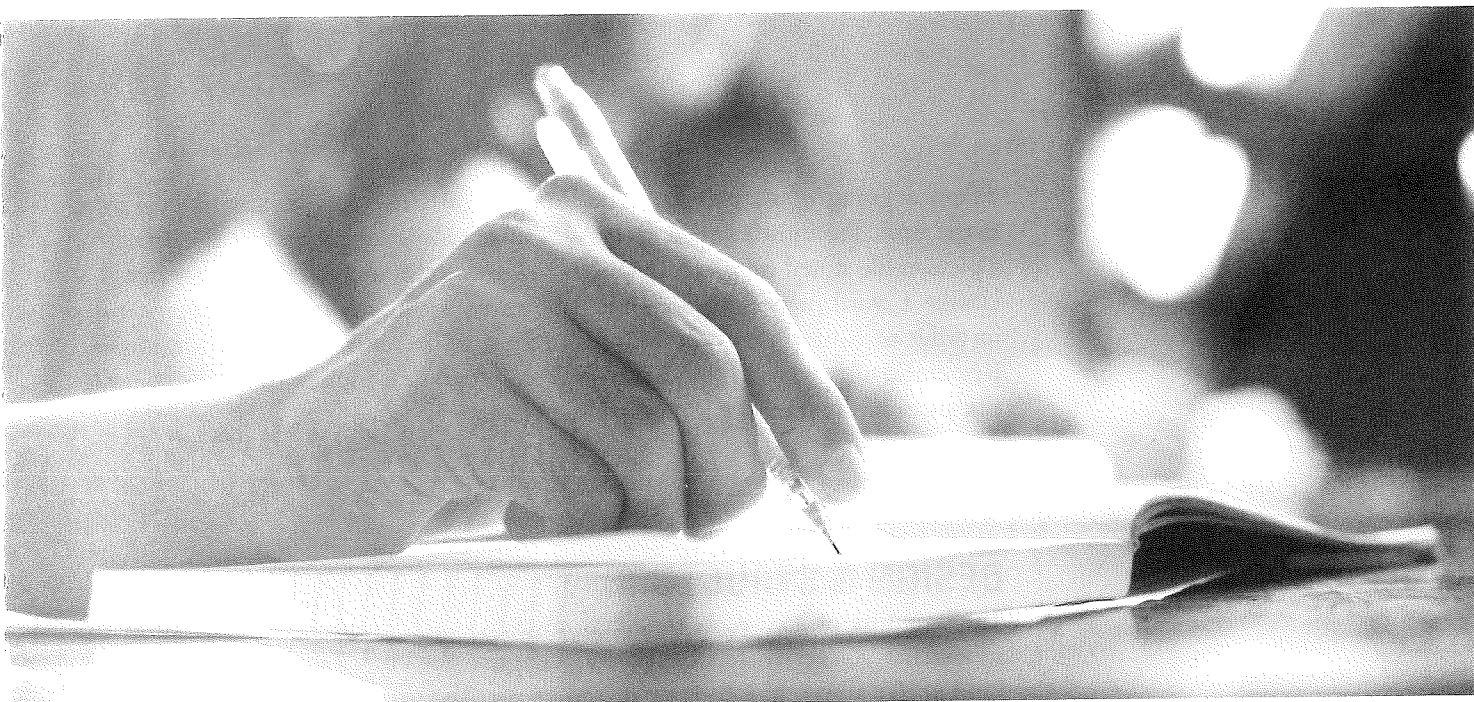
```
III. int x = 10;
    int y = 20;
    int z = 0;
    if (x < y || 10 < y/z)
        System.out.println("Homer");
    else
        System.out.println("Bart");
```

Which of the code segments above will run without error?

☐ (A) I only☐ (B) II only☐ (C) III only☐ (D) II and III

Chapter 6 Summary

- While statements can be used to cause a command or series of commands to execute while a certain boolean statement is true.
- For statements are similar to while statements but include the initialization of a variable before the looping and a modification of the variable after each execution of the loop.



Chapter 7

Writing Classes

When a group of statements, including control structures, is assembled into a single unit, the unit is called a **class**. Similar to a word processing document or a picture file stored on your computer, a class is stored on your computer in a file. Unlike a word processing document, however, a class must follow specific rules in order to conform to Java and, as a result, to be understood by the compiler. Remember that your brilliantly constructed program is useless unless the compiler can successfully translate it into machine code.

DESIGN & STRUCTURE

The structure of a class, at this level anyway, is straightforward. Take all of the statements you've written before this chapter, put curly brackets around them, kind of like a huge block, name it, and save it. The naming conventions for a class are similar to those of a variable, except that they should begin with a capital letter. The name of the class should reference the purpose or function of the class. For example, a class that calculates a GPA might be called `GPA` or `GradePointAvg`. The way we declare a class in Java is using the header:

```
public class GradePointAvg
{
    // statements not shown
}
```

Note that the statements in the class reside at least one tab stop across the page. If the programmer would like to use more white space within the structures of the class, they will be tabbed further into the page:

```
public class GradePointAvg
{
    // other code and variable declarations not shown

    while (/* condition not shown */)
    {
        // looped statements not shown
    }
}
```

Typically, variable declarations are placed into the code first, right after the class header, as a sort of “setup” for the rest of the class. Of course, this is not a rule, but it is a good guideline in order to keep your code readable.

This class would be saved to the computer as `GradePointAvg.java`; the `.java` extension helps the compiler to recognize the file as containing Java code. If the code compiles (there are no syntax errors), the compiler will create another file in the same folder called `GradePointAvg.class`; this file is unreadable to the programmer because it is encoded in machine language. The computer, however, uses this file to execute the program. The `.java` file is called the **source code** of the program, since it defines the program's actions and functions.

Well, Actually...

This is a convention but not a rule. You can have all of the code on a single line if your heart so desires, but that would be very bad form.

Remember that classes are executed top-down—i.e., line 1 followed by line 2, etc. If programmers want to alter top-down execution, they must use a flow control structure, as we studied before.

1 Mark for Review

Which of the following class declarations would cause a compile-time error? Assume the rest of the code compiles as intended.

- (A) `public class Calculus`
- (B) `public class apCalculus`
- (C) `public class APCalculus`
- (D) `public class 4APCalculus`

Here's How to Crack It

Choices (A) and (C) follow the format as discussed above. The naming convention for class names is that they *should* begin with a capital letter; however, this is not required by the compiler, so (B) does not cause an error. The real problem with (D) is that it begins with a numerical character, which is not allowed in a class name. The answer is (D).

POE is a great tool, but it isn't always enough on an AP Exam.
Know your stuff!

METHODS

Picture a Java class as a closet, with essentially limitless possibilities. If you were to declare a single shelf to hold, say, board game boxes...that would be a great way to organize your collection of games, toys, and other sophisticated possessions.

A class is organized in this way using methods.

A **method** is a group of code that performs a specific task.

Although a class can simply contain a list of commands, any program that accomplishes a task could easily balloon to 100 lines of code or more. Instead of writing a class that contains a giant list of disorganized code, writing separate methods for each small task (within the overarching task) is an excellent way to make your code more readable. In addition, method-based classes enable easier flow control.

To understand this concept, let's break down a bright, sunny Saturday in the middle of the summer (don't worry—you will make it). Perhaps some of the activities you would do might include getting out of bed (obviously), eating breakfast, taking a shower, getting ready for the beach, driving to the beach, enjoying the beach, and eating lunch. Each of these activities has its own specific components; for example, part of eating breakfast includes drinking orange juice and taking a daily vitamin supplement. The pseudocode of a program that simulates this Saturday's activities might look like this:

```
public class Saturday1
{
    hear alarm;
    turn off alarm;
    get out of bed;
    make breakfast;
    eat breakfast; ...
}
```

You get the idea... a long list of commands that occur top-down. Now, consider each of these general activities as a separate grouping of smaller activities, called a method. Each small activity will be defined later, but your Saturday class would look much cleaner:

```
public class Saturday2
{
    wake up method; // includes alarm
    eat breakfast method; // includes preparation, OJ,
                        // vitamin, etc.
    take a shower method; // includes preparing for the
                        // shower, etc.
    beach method; // includes prep for the beach, driving
                  // to the beach, etc.
    eat lunch method;
}
```

The result is a cleaner, more structured class that is easier to read. A class that is created to control a larger program, such as `Saturday2`, is called a **driver class** because it *drives* the program through its general structure which will, in turn, execute the smaller commands. Note also that if, for some reason, the programmer wanted the simulation to take a shower before eating breakfast, the two lines would be switched in `Saturday2` and the job would be done. Performing the same change in `Saturday1` would involve switching multiple lines of code. Even more poignant would be a simulation that repeats method calls, such as one that represents your extra-hungry older brother, who eats breakfast twice. The driver would simply call the breakfast method twice (or use a loop to call it more than twice) and the job would be completed.

An **object class** is a different kind of class which houses the “guts” of the methods that the driver class calls. As you understand this setup, you will see that your driver classes begin to shorten, while your object classes expand. For example, the driver of a car truly performs a relatively small set of actions in order to drive the car: start the engine, buckle the seatbelt, check the mirrors, accelerate, brake, steer. The driver does not actually understand (nor does he need to) exactly how these operations work: he just does them. How does the accelerator pedal *actually* make the car move more quickly? Doesn’t matter. The “driver” just operates the pedal, and the rest happens under the hood. The object class is the “under the hood” class. It defines all of the aspects of an object; more specifically, it defines what the object *has* and what the object *does*.

The object class referred to in this section should not be confused with the `Object` class in the Java language, which is the Parent class of all classes. The `Object` class is part of the `Java.lang` package. Every class in Java is a descendant of this class. Every class in Java inherits the methods of the `Object` class.

For the AP Exam, you must be able to write, troubleshoot, and understand object AND driver classes, as well as understand how they interact. These skills are HUGE in the free-response questions!

Back to the breakfast example...

Pseudocode for the object class might look something like this:

```
public class Saturday3
{
    wake up(...)
    {
        hear alarm;
        turn off alarm;
        get out of bed;
    }

    breakfast(...)
    {
        make breakfast;
        eat breakfast;
    }
}
```

Since our object class could be used for any day of the week, not just Saturday, `BeachDay` might be a more appropriate name for this class. The driver class from before, which executes a series of actions for this particular day, might be called `Saturday`.

2 Mark for Review

Which of the following statements would best describe an efficient design to represent a pair of sunglasses?

- (A) Three classes: `UnfoldGlasses`, `CleanGlasses`, and `WearGlasses`
- (B) An `UnfoldGlasses` class with methods `CleanGlasses` and `WearGlasses`
- (C) A `PairOfSunglasses` class with boolean variables `unfolded`, `cleaned`, and `worn`
- (D) A `PairOfSunglasses` class with methods that `unfold`, `clean`, and `wear` the objects in the class

Here's How to Crack It

Design questions must account for efficiency and “beauty,” since the former isn’t formally tested on the multiple-choice questions and the latter is not tested in the free-response questions. Since a pair of sunglasses is best represented as an object—it has attributes and can do tasks—eliminate (A) and (B). Since the tasks of a `PairOfSunglasses` object would best be performed in a method, (C) is not appropriate. The answer is (D).

One more item that needs to be mentioned...do you see how each method in the object class has a “title,” or **header**? The header is used to indicate the overall function of a method.

This is a Java book, right? So let’s look at some Java. These two classes are designed to simulate a phone downloading and opening a new app.

```

1 import Phone;
2 public class UsePhoneApp
3 {
4     public UsePhoneApp() // just do it, we'll explain later
5     {
6         Phone myPhone = new Phone();
7         myPhone.downloadApp();
8         myPhone.openApp();
9         myPhone.closeApp();
10    }
11 }

1 public class Phone
2 {
3     private boolean hasApp;
4     public Phone()
5     {
6         hasApp = false;
7     }
8     public void downloadApp()
9     {
10        hasApp = true;
11    }
12
13    public void closeApp() // closes the app
14    {
15        if (hasApp)
16            System.out.println ("App is closed.");
17    }
18
19    public void openApp() // opens the app
20    {
21        if (hasApp)
22            System.out.println ("App is running...");
23    }
24 }

```

OUTPUT:

```

App is running...
App is closed.

```

There are many components in this program; first, note that we now have two classes that make up a single program. In isolation, neither of these classes would produce a useful result. The `UsePhoneApp` class would not compile because the

compiler does not understand the methods if they are not present (compiler: what does `openApp` mean?); the `Phone` class would compile but would not actually perform any actions without `UsePhoneApp` (compiler: when do I `openApp`, and on which phone?). In order to allow the `UsePhoneApp` class to access the `Phone` class, the line

```
import Phone;
```

must be used at the top of the file above the class declaration.

Lines 4–7 in `Phone` make up a special kind of method called a **constructor**. An object must be constructed before it can perform any actions, just as a car must be built before it can be driven. Line 6 in `UsePhoneApp` calls this constructor method, which “builds” the `Phone` object; this command must occur before the non-constructor methods are called; otherwise, the compiler will return an error. Imagine someone pressing an imaginary accelerator pedal and trying to drive somewhere without a car. Interesting, but not possible.

Line 3 of `Phone` defines whether the `Phone` has an app (true) or does not (false). This is the sole attribute of a `Phone` object in our program; more complex programs may have dozens or even hundreds of attributes (a phone, in reality, has many more attributes, such as a screen, volume buttons, etc.). The programmer should write these attributes, called **instance variables** or **fields**. Following the data fields is the constructor, as we stated before, and then a series of methods that control what the `Phone` *can do*.

An object class defines what an object HAS and DOES.

A method has several components. The method header is built in a very specific way; its syntax is as follows:

```
visibility returnType methodName (param1, param2, ...)
```

The visibility of a method can be public or private, depending on the situation (more on this later). The **return type** identifies what type of data, if any, will be returned from the method after its commands are executed. The name of a method, similar to the name of a variable, should be an intuitive name that summarizes the function of the method; it should conform to the same specifications as a variable name (begin with a lowercase letter, etc.). The **parameters**, which are optional, are the data that the method needs in order to perform its job.

In our previous example, the method header for `closeApp` was:

```
public void closeApp()
```

This is functional, but it is also relatively simple for the example. Its visibility is public, its return type is void (there is no information returned), and it does not have parameters. A more realistic method to close an app on a phone might

require information regarding when it should be closed, and might return data, such as whether or not it was closed successfully. Thus, the revised method header might look like

```
public boolean closeApp (int minutes)
```

Note that the single parameter, `minutes`, is defined with its type. Since `minutes` will be a temporary variable that exists only during the execution of this method, it must be defined as `new`, right in the header. A method can have any number of parameters, depending on the programmer's design decisions for the program.

A method must begin with a header containing a visibility modifier (private or public), a return type or void if no information will be returned, and a method name. Parameter(s) are optional, depending on the method's requirements to perform its task.

The ultimate purpose of a method is to perform some sort of task with respect to the object. Note that `openApp()` and `closeApp()` simply access the data field `hasApp` and react accordingly; i.e., the message is displayed only if the value `hasApp` is `true`. The method `downloadApp()` is more profound, in a way; rather than simply accessing data, it actually changes the value of a data field—in this case, updates `hasApp` to `true` once the “app” is “downloaded.” As a result, it is common in Java to label methods like `openApp()` and `closeApp()` as **accessor methods** and to label methods like `downloadApp()` as **mutator methods**.

We have used an analogy to a car several times already in this chapter, but it's a great way to understand these concepts. When you want to start a car, you have to go through a series of steps. Since all of those steps perform, ultimately, a single action—the car starting—a method would be a great way to keep the “car starting” commands in a single, convenient unit. As a programmer, you would have to decide what data the method would need in order to work (parameter(s)) and what data, if any, the method would return. Does the car need a key to start? A password code? Does the brake pedal have to be depressed? This information would all be accepted through parameters. Should the method return whether the car has started? How long it took to start? A code that represents whether it needs an oil change? These are all examples of possible return data.

In some cases, there may be a situation in which the car can be started multiple ways—for example, the owner has a push-button starter and a remote starter. The designer could write two `startCar` methods (or three, or four...), each one performing the same task but requiring different information (parameters) to do the task. This process is called **overloading** and can be accomplished by writing two or more methods with identical names but different types and/or numbers of parameters.

Precondition

A **precondition** is a comment that is intended to inform the user more about the condition of the method and guarantees it to be true.

On the AP Exam, the precondition shows up as a comment immediately above the method. It is the responsibility of the program that calls the method not to pass parameters that violate the precondition.

```
/** precondition—a and b are positive integers
 *
 * public int sum(int a, int b)
 *
 * {
 *
 * }
```

Postcondition

A **postcondition** is a condition that must always be true just after the execution of some section of code or after an operation in a formal specification.

On the AP Exam, the postcondition is written as a comment either before or within the method as shown in the example below. The method designer is responsible for making sure that these conditions are met.

```
public int sum100(int a, int b)
{
<code>

// postcondition—returns 100, if sum is greater than 100, or the value of sum

if (sum < 100)

    return sum;

else

    return 100;}
```